

120 minutes. Tous Documents papier autorisés. Barème à titre indicatif.

1 Esterel

On modélise un système de contrôle des entrées-sorties d'un musée :

- le musée dispose d'un nombre arbitraire d'entrées et de sorties devant lesquelles se forment des files de visiteurs,
- à chaque entrée (resp. sortie) correspond un capteur qui vaut vrai quand une personne s'ajoute à la file de cette entrée (resp. sort par cette sortie) et faux sinon. On suppose que la granularité des instants est telle qu'une personne au plus rejoindre une file à chaque instant.
- une entrée (resp. sortie) peut être ouverte ou fermée.
- quand une entrée (resp. sortie) est ouverte, si au moins une personne est dans la file associée, alors on fait entrer (resp. sortir) une personne tous les K instants (cette personne quitte donc la file) (K est une constante arbitraire).
- les normes de sécurité fixent un nombre maximum N de personnes pouvant visiter simultanément le musée,
- lorsque la limite N est atteinte (ou dépassée) on bloque les entrées (des personnes peuvent toujours arriver dans la file, mais ne peuvent pas passer l'entrée/sortie),
- lorsque l'alarme du musée retentit, les entrées et les sorties sont temporairement fermées.

Question 1. Ecrire le modèle en Esterel en suivant ces consignes :

- on écrit un unique module générique pour les entrées (respectivement pour les sorties), associé (à l'initialisation) à un capteur.
- on écrit un module de contrôle qui compte le nombre de personnes actuellement à l'intérieur du musée et qui ferme les entrées le cas échéant
- le système doit pouvoir réagir à un signal extérieur d'alarme, fermant entrées et sorties, et un signal extérieur de *reset* qui réouvre les entrées et sorties.
- l'initialisation du système se fera avec 5 capteurs (3 entrées, 2 sorties)

Question 2. Ajouter au système un module qui calcule à chaque instant, le nombre moyen de visiteur depuis l'ouverture du musée (c'est à dire la somme des visiteurs à chaque instant divisée par le nombre d'instants).

Question 3. Ajouter un module qui ouvre une nouvelle une entrée lorsque le nombre de personnes dans les files d'entrées est plus grand que M .

Question 4. Modifier le code pour que l'on ferme une entrée lorsque le nombre de personnes dans les files d'entrées est nul, et qu'au moins deux entrées actuellement sont ouvertes.

2 Web

On cherche à développer une *plate-forme d'évaluation en ligne* comprenant les fonctionnalités suivantes :

- des enseignants peuvent proposer des Questionnaires à Choix Multiples sur la plateforme.
- des étudiants peuvent s'évaluer en participant à un QCM.
- chaque étudiant possède un score basé sur sa réussite aux différents QCM auxquels il a accès.
- l'enseignant qui crée un QCM décide 1) de la liste des étudiants qui y auront accès, 2) de la fenêtre temporelle dans laquelle le QCM est utilisable, 3) du barème.
- la page d'accueil de l'application pour les étudiants propose la liste des QCM disponibles pour cet étudiants, la liste des QCM déjà faits, le score de l'étudiant et le tableau des scores des meilleurs étudiants de l'application.

Le but de cet exercice est de réaliser un mini-dossier décrivant l'implémentation de cette application web en respectant les points suivants:

- l'application est composée d'un unique client pour les étudiants et les enseignants et d'un unique serveur.
- Le serveur Web est écrit en Servlets purs ou en Go (sans utiliser de *framework* de développement).
- le client et le serveur communiquent selon une API REST.

Le dossier comprend:

1. **Schéma:** un **schéma général** de l'application, détaillant les composants internes coté client et côté serveur, les différentes communications entre les composantes de l'application, et une répartition des fonctionnalités.
2. **Use Cases:** une série de plusieurs *use cases* décrivant **textuellement** (en prose) des exemples d'utilisations caractéristiques de l'application.
3. **DB:** une description des bases de données, des différentes tables/documents et de leur composition ainsi que quelques exemples d'entrées.
4. **REST:** une description complète de l'API REST mise en oeuvre par le serveur, des mots-clefs utilisés, des requêtes possibles, ainsi que plusieurs exemples d'utilisation (requete HTTP, reponse HTTP).
5. **Servlets/Handlers:** un plan (noms, signatures, rôles) des Servlets/Handlers présents dans le serveur de l'application, de leurs liens avec les mots-clef de l'API ou les services.
6. **Servlet:** le plan du code d'un Servlet ou d'un *handler* Go caractéristique, au choix.
7. **Client:** le plan du site pour les usager avec les différents écrans proposés (on peut choisir un client monopage), et une description des contenus proposés et la mise en avant des appels asynchrones (AJAX) effectués par le client.
8. **Javascript:** le code JS de la partie du client chargée d'afficher le tableau des scores (en le maintenant à jour).

3 Lustre

Ecrire en Lustre :

1. un noeud qui prend en entrée un flot d'entiers $(x_n)_{n \in \mathbb{N}}$ et qui renvoie en sortie un flot d'entiers $(y_n)_{n \in \mathbb{N}}$ défini par $y_0 = x_0$, $y_1 = \frac{y_0 + x_1}{2}$ et $y_{n+2} = \frac{y_{n+1} + y_n + x_{n+2}}{3}$ pour tout $n \in \mathbb{N}$.
2. un noeud qui prend en entrée un flot d'entiers $(x_n)_{n \in \mathbb{N}}$, un flot de booléens $(b_n)_{n \in \mathbb{N}}$ et qui renvoie un flot d'entier $(y_n)_{n \in \mathbb{N}}$ dont le comportement est donnée par :

quand b est faux, x et y sont égaux; quand b devient vrai, y devient un flot constant dont la valeur est celle de x au moment où b est devenu vrai, quand b redevient faux; y reprend son comportement initial

4 Canaux Synchrones

On veut écrire un *Jeu de la vie de Conway désynchronisé* en Go :

- Le jeu se déroule sur une grille en deux dimensions $N \times N$.
- Dans chaque case de la grille se trouve une "cellule" qui peut être dans deux états (mutuellement exclusifs): morte ou vivante.
- Chaque cellule peut accéder aux informations situées dans les 8 cases voisines (au dessus, en dessous, à gauche, à droite et dans les quatre diagonales). Les cellules sur les bords de la grille (et dans les coins) ont moins de 8 voisines.
- Si une cellule morte a exactement trois voisines vivantes, elle devient vivante, sinon elle reste morte.
- Si une cellule vivante a deux ou trois voisines vivantes, elle reste vivante, sinon elle devient morte.

Dans le jeu de la vie *classique* on calcule *simultanément* l'état suivant de toute la grille à partir de son état courant. Ainsi toutes les cellules vérifient leur conditions *en même temps*.

Dans notre variante, chaque cellule est initialisée avec un *cycle*, un entier $cycle[i, j]$, différent et examine ses 8 voisines toutes les $cycle[i, j]$ millisecondes.

Question 1. Ecrire le code correspondant au jeu de la vie désynchronisé (on demande pas de faire l'affichage) en Go, en respectant les consignes suivantes :

- une goroutine s'occupe de la mise-à-jour de chaque cellule,
- aucune information n'est transmise par *mémoire partagée* (i.e. les goroutines utilisent des canaux pour interroger le statut de leurs voisines)

Question 2. Mettre-à-jour le code en y ajoutant une goroutine d'affichage : à intervalles réguliers, elle peut geler le jeu (elle arrête le cycle de toutes les cellules), afficher la grille sur la sortie standard, puis dégeler le jeu (les cellules reprennent leur cycle).